UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

PROGRAMACIÓN 1

Trabajo Práctico Integrador

Sistema de Control de Inventario

Ferretería – Python: lista de diccionarios, funciones y manejo de errores

Estudiante	Oscar Hernán Leon Oyarzo
Comisión	9
Coordinador	Alberto Cortez
Tutor	Neyén Bianchi
Fecha de entrega	Noviembre de 2026
Archivo de inicio	main.py

1. Introducción y objetivo

Este trabajo consiste en un sistema de consola, escrito en Python, para controlar el inventario de una ferretería. El usuario puede cargar herramientas con su stock inicial, ver el inventario completo, consultar el stock de una herramienta, obtener un reporte de las agotadas, dar de alta un producto nuevo y registrar ventas o ingresos de mercadería. El programa se maneja con un menú que se repite hasta que el usuario elige salir.

El objetivo fue aplicar en un caso concreto los contenidos de la materia: listas, diccionarios, funciones, condicionales, bucles y manejo de errores con **try/except** y **raise**, respetando las reglas técnicas del enunciado: **inventario** es la única colección de datos, no se usan clases ni variables globales y cada opción del menú del 1 al 6 tiene su propia función.

Antes de escribir el código armé el diseño: primero el esqueleto del menú con las seis funciones vacías, después las dos funciones auxiliares que se repiten en varias opciones y, por último, cada opción por separado. Cada parte se probó antes de pasar a la siguiente (ver sección 5).

2. Estructura del programa y cómo ejecutarlo

El enunciado permite dividir el programa en varios archivos .py propios. Lo organicé en un archivo principal y un paquete con las funciones, para que cada archivo tenga una responsabilidad clara:

Archivo	Contenido	Responsabilidad
main.py	mostrar_menu(), main() y la llamada main()	Punto de entrada. Crea inventario, muestra el menú y deriva a cada opción.
paquete_funciones/__init__.py	Vacío	Indica a Python que la carpeta es un paquete importable.
paquete_funciones/auxiliares.py	validar_nombre(), buscar_herramienta()	Reglas que usan varias opciones: limpiar/validar nombres y buscar sin distinguir mayúsculas.
paquete_funciones/opciones.py	Las 6 funciones de las opciones 1 a 6	Lógica de cada opción del menú.

Archivo que se ejecuta para iniciar el programa: main.py. Se abre la carpeta *TP INTEGRADOR FINAL* en Visual Studio Code y se ejecuta main.py con el botón ▶, o desde la terminal, ubicado en esa carpeta, con `python main.py`. Es importante ejecutarlo desde la carpeta raíz, porque los imports usan la ruta `paquete_funciones.opciones` y `paquete_funciones.auxiliares`.

```
from paquete_funciones.opciones import (cargar_herramientas,mostrar_inventario,
                                        consultar_stock,reporte_agotados,alta_producto,actualizar_stock)
```

main.py, líneas 3–4: main.py importa por nombre las seis funciones del paquete.

La carpeta __pycache__ la genera Python automáticamente al importar el paquete (guarda una versión compilada para acelerar el arranque); no forma parte del código fuente.

3. Conceptos aplicados

En cada concepto explico dónde y por qué lo usé en mi programa.

3.1 Listas

La única colección del programa es la lista **inventario**. Se crea vacía dentro de `main()` (no es global) y se pasa como parámetro a las seis funciones de las opciones. Uso una lista porque el inventario es una colección **ordenada** que crece: cada herramienta nueva se agrega al final con append, y para listar, buscar o contar agotados alcanza con recorrerla con un `for`.

```
def main():
    inventario = []
    opcion = 0
```

main.py, líneas 17–19: la lista se crea dentro de main(), sin variables globales.

Como la lista es **mutable** y se pasa por referencia, cuando `cargar_herramientas` o `alta_producto` hacen `inventario.append(...)`, modifican la misma lista que está en `main()`. Por eso ninguna función necesita devolverla con `return`. Para saber si está vacía uso `len(inventario) == 0` (opciones 2, 3, 4 y 6).

3.2 Diccionarios

Cada herramienta es un diccionario con exactamente dos claves: **'herramienta'** (el nombre) y **'cantidad'** (el stock). Elegí un diccionario porque permite acceder a cada dato por su nombre, por ejemplo `herramienta['cantidad']`, en lugar de recordar posiciones. Tal como exige el enunciado, el diccionario se arma **directamente dentro del append**, sin crear un diccionario suelto antes:

```
        inventario.append({'herramienta': nombre, 'cantidad': stock})
```

opciones.py, línea 38 (opción 1); la misma forma se usa en la línea 81 (opción 5).

En la opción 6 la variable `herramienta` apunta al diccionario que ya está dentro de `inventario` (no es una copia). Así, `herramienta['cantidad'] -= unidades` modifica el stock guardado en la lista. Este tipo de variable que apunta a un elemento existente está permitida por el enunciado.

3.3 Funciones

El programa está dividido en funciones con una tarea cada una. Las opciones 1 a 6 tienen su función propia y reciben `inventario` como parámetro; la opción 7 se resuelve en `main()`. Los nombres usan snake_case.

Función	Archivo	Tarea
mostrar_menu()	main.py	Imprime las 7 opciones. No recibe parámetros porque no usa el inventario.
main()	main.py	Crea inventario, repite el menú, llama a cada opción y captura los errores.
validar_nombre(nombre)	auxiliares.py	Quita espacios con strip(); si queda vacío lanza ValueError; si no, devuelve el nombre limpio.
buscar_herramienta(inventario, nombre)	auxiliares.py	Recorre la lista y compara en minúsculas; devuelve el diccionario o None.
cargar_herramientas(inventario)	opciones.py	Opción 1: carga múltiple con reintentos dentro de la función.

Función	Archivo	Tarea
mostrar_inventario(inventario)	opciones.py	Opción 2: lista todas las herramientas y su stock.
consultar_stock(inventario)	opciones.py	Opción 3: busca una herramienta y muestra su stock.
reporte_agotados(inventario)	opciones.py	Opción 4: lista solo las de stock 0, con mensaje propio si no hay.
alta_producto(inventario)	opciones.py	Opción 5: agrega una herramienta; los errores vuelven a main().
actualizar_stock(inventario)	opciones.py	Opción 6: venta o ingreso, siguiendo los 7 pasos en orden.

Separé `validar_nombre` y `buscar_herramienta` porque las opciones 1, 3, 5 y 6 necesitan exactamente la misma regla. Así la regla está escrita una sola vez y, si cambia, se corrige en un solo lugar.

```
def buscar_herramienta(inventario, nombre):
    for herramienta in inventario:
        if herramienta['herramienta'].lower() == nombre.lower():
            return herramienta
    return None
```

auxiliares.py, líneas 10–14: el return dentro del for sale apenas encuentra la herramienta; return None solo se alcanza si no la encontró.

3.4 Condicionales

Uso `if / elif / else` para decidir el camino del programa:

- En `main()` (líneas 24–39), una cadena de `if/elif` deriva cada número a su función; el `else` final lanza el error de opción fuera de rango.
- Para los **casos informativos** uso una condición y un mensaje, sin excepción, como pide el enunciado: inventario vacío (`len(inventario) == 0`), herramienta inexistente (`herramienta == None`) y tipo de movimiento inválido (`tipo != '1' and tipo != '2'`).
- En la opción 6 el tipo de movimiento se compara **como texto** ('1' o '2'), sin `int()`, para que una entrada como "abc" se resuelva con el `if` y un mensaje.

```
    tipo = input('Tipo de movimiento (1 = venta, 2 = ingreso): ').strip()
    if tipo != '1' and tipo != '2':
        print('Tipo de movimiento inválido. Debe ser 1 o 2.')
        return
```

opciones.py, líneas 94–97: condición + mensaje + return, sin raise.

En `actualizar_stock` usé `return` después de cada control que falla, en lugar de anidar varios `else`. Así la función queda en línea recta y sigue el mismo orden de pasos del enunciado, y nunca se piden datos para una herramienta inexistente o un movimiento ya rechazado.

3.5 Bucles

- **while del menú** (main.py, línea 20): `while opcion != 7` repite el menú después de cada operación; solo la opción 7 lo termina. `opcion` arranca en 0 para que el bucle entre la primera vez.
- **while de reintento** (opción 1, líneas 10, 21 y 31): repiten el pedido de la cantidad, del nombre y del stock hasta que el dato sea válido. Los valores iniciales están elegidos a propósito: la cantidad empieza en 0, el nombre en '' y el stock en **-1**, porque 0 es un stock válido.

- **for con range** (línea 18): `for i in range(cantidad_a_cargar)` repite la carga tantas veces como herramientas indicó el usuario. Muestro `i + 1` porque range empieza en 0.
- **for sobre la lista** (opciones 2 y 4 y buscar_herramienta): recorre `inventario` elemento por elemento. En el reporte de agotados uso un **contador** numérico (`agotados += 1`) en vez de otra lista, y recién al terminar el `for` sé si debo mostrar "No hay herramientas agotadas".

```
    for i in range(cantidad_a_cargar):
        print(f'\nHerramienta {i + 1} de {cantidad_a_cargar}')
        nombre = ''
        while nombre == '':
            ...
        stock = -1
        while stock < 0:
            ...
        inventario.append({'herramienta': nombre, 'cantidad': stock})
```

opciones.py, líneas 18–38: un while para el nombre y otro para el stock; si falla el stock se vuelve a pedir solo el stock y se conserva el nombre.

3.6 Manejo de errores con try/except y raise

Uso los dos mecanismos que pide el enunciado, cada uno para un tipo de situación:

- **int() produce el ValueError solo** cuando el texto no es un entero (por ejemplo "abc" o "3.5"). No hace falta escribir un raise.
- **raise ValueError('motivo')** lo escribo yo para las **reglas de negocio**: nombre vacío o duplicado, stock negativo, cantidad o unidades no mayores que 0, venta mayor al stock y opción fuera de rango.

Lo más importante del diseño es **dónde** se captura cada error:

Dónde	Qué pasa	Por qué
Dentro de cargar_herramientas (opción 1)	El try/except está dentro de cada while: se muestra el motivo y se vuelve a pedir el mismo dato.	La opción 1 debe reintentar sin volver al menú y conservar los datos válidos.
En main() (opciones 3, 5 y 6 y el menú)	La función no tiene try: el error sale de ella y lo atrapa el except de main(), que imprime el motivo y vuelve al menú.	Estas opciones deben cancelar la operación sin agregar ni modificar nada.

```
        except ValueError as e:
            print(f'Error: {e}')
```

main.py, líneas 40–41: un solo except atiende la opción no numérica, la opción fuera de rango y los errores que salen de las opciones 3, 5 y 6.

Como el `raise` corta la función en ese mismo momento, en las opciones 5 y 6 el append o la modificación del stock solo se ejecutan si pasaron todas las validaciones: ningún dato inválido deja el inventario a medias. No usé clases de excepción propias porque las clases están prohibidas; alcanza con `ValueError` y un mensaje descriptivo.

4. Diagramas de flujo

Los diagramas reflejan mi código. Los colores indican: azul claro, entrada o proceso; amarillo, decisión; verde, la operación correcta; rojo, errores con ValueError; naranja, mensajes informativos sin excepción.

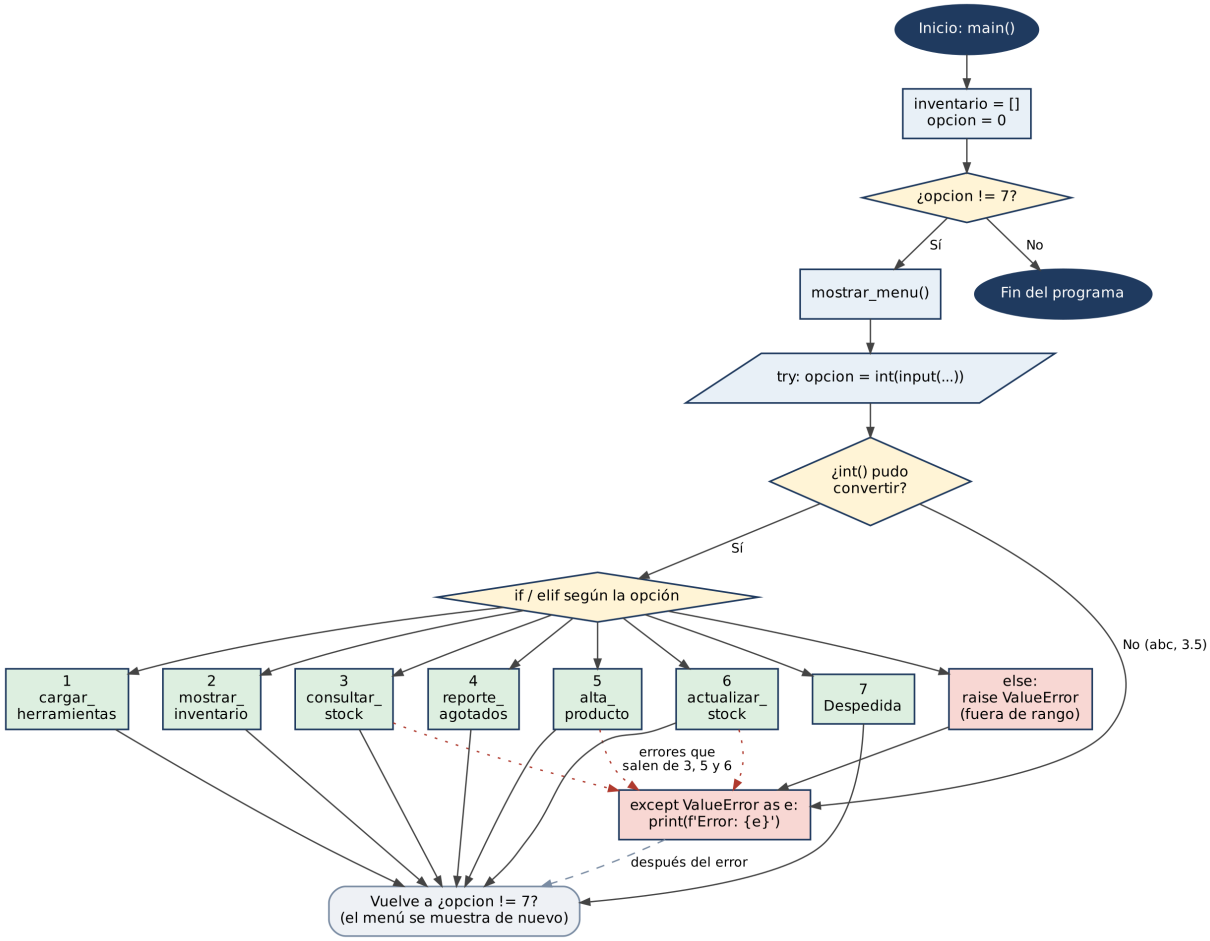


Figura 1. Menú principal: main() con el while, la derivación a las opciones y el except que vuelve al menú.

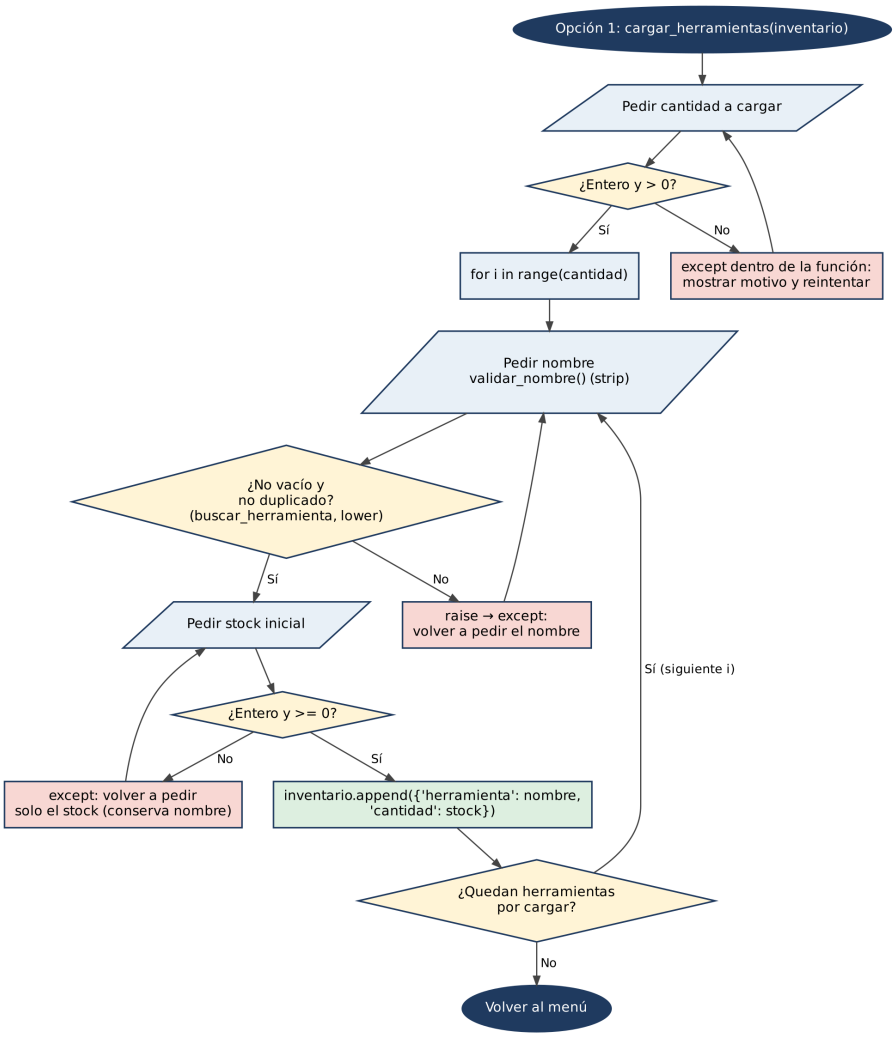


Figura 2. Recorrido de carga (opción 1): cada dato se valida y se reintenta dentro de la función.

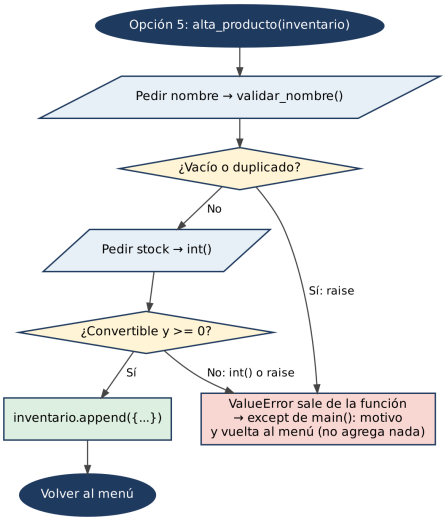


Figura 3. Validación de datos en el alta (opción 5): el error sale de la función hacia main().

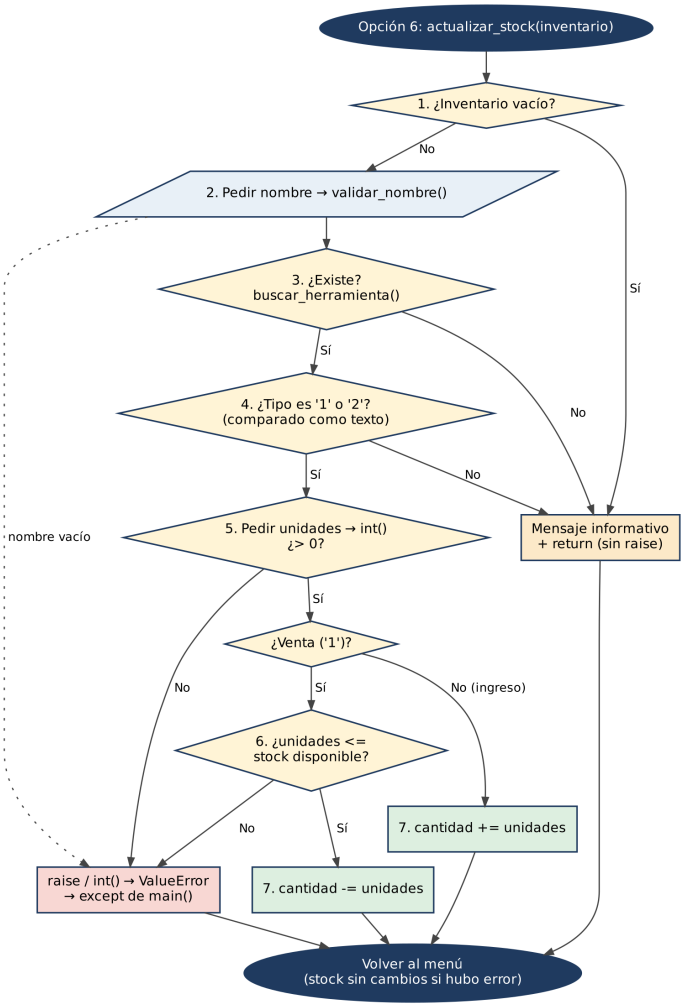


Figura 4. Actualización de stock (opción 6): los 7 pasos en el orden del enunciado.

5. Pruebas realizadas

Probé cada opción apenas la terminé, cubriendo los casos correctos y las situaciones de la tabla de validaciones del enunciado (sección 5). En el anexo están las capturas de la terminal.

#	Opción	Entrada	Resultado esperado	Resultado obtenido
1	Menú	abc	Error de conversión y vuelve el menú	OK
2	Menú	9	Error: Opción fuera de rango (1 a 7).	OK
3	Menú	7	Despedida y fin del programa	OK
4	1	Cantidad abc, luego 0	Error y vuelve a pedir la cantidad	OK
5	1	Nombre vacío	Error: el nombre no puede estar vacío; reintenta	OK
6	1	Stock -3, luego x	Error y vuelve a pedir solo el stock (conserva "Martillo")	OK
7	1	"martillo" ya cargado como "Martillo"	Error de duplicado (ignora mayúsculas); reintenta	OK
8	1	Herramienta con stock 0	Se acepta (0 es válido)	OK
9	1 (2ª vez)	Nueva carga	Se agregan a las ya cargadas	OK
10	2	Inventario vacío / con datos	Mensaje propio / listado completo	OK
11	3	Inventario vacío	Mensaje, sin pedir el nombre	OK
12	3	Nombre vacío	Error y vuelve al menú (except de main)	OK
13	3	MARTILLO / Serrucho	Stock de Martillo: 5 / "Serrucho" no está en el inventario	OK
14	4	Vacío / sin stock 0 / con stock 0	Tres resultados distintos; lista solo las de stock 0	OK
15	5	Nombre vacío	Error, sin pedir el stock	OK
16	5	Stock abc / -2	Error y vuelve al menú, no agrega nada	OK
17	5	TALADRO (ya existe)	Error de duplicado, sin pedir el stock	OK
18	6	Inventario vacío / Serrucho	Mensaje, sin pedir más datos	OK
19	6	Tipo abc	Tipo inválido, sin pedir unidades	OK
20	6	Unidades 0 / vender 8 con stock 6	Error; "Stock insuficiente: hay 6 unidades."; el stock no cambia	OK
21	6	Vender 6 / ingresar 10	Stock 0 (aparece en agotados) / stock 10	OK

6. Lenguaje, entorno y recursos utilizados

Python 3.13 (64 bits). Python es un lenguaje de programación interpretado, de alto nivel y de sintaxis clara, mantenido por la Python Software Foundation. Usé la versión 3.13, instalada desde python.org, porque es la que usamos en la cursada y trae todo lo necesario (listas, diccionarios, funciones, excepciones) sin instalar librerías externas. El programa solo usa funciones incorporadas: input, print, int, len, range, strip() y lower().

Visual Studio Code 1.145.2. Es un editor de código gratuito de Microsoft, con extensiones para cada lenguaje. Lo elegí porque es el entorno recomendado en la materia y, con la extensión de Python, resalta la sintaxis, marca la sangría con guías verticales, muestra advertencias (por ejemplo, funciones usadas antes de importarlas) y permite ejecutar main.py con un clic en la terminal integrada.

Terminal integrada (PowerShell, Windows). Es la consola dentro de VS Code donde se ejecuta el programa y se interactúa con el menú. La usé para todas las pruebas de la sección 5, porque muestra la entrada y la salida igual que las verá el usuario.

Paquetes y módulos de Python. Investigué cómo dividir un programa en varios archivos: una carpeta con un archivo `__init__.py` se convierte en un paquete, y sus archivos .py (módulos) se importan con `from paquete.modulo import funcion`. Lo elegí para separar el menú, las auxiliares y las opciones.

Asistente de inteligencia artificial (Claude, de Anthropic). Lo consulté como apoyo de estudio, para aclarar dudas sobre los conceptos y revisar el código mientras avanzaba, de forma parecida a una consulta de tutoría. Las pruebas las hice yo en mi equipo, y cada parte del programa la puedo explicar.

Bibliografía y referencias

- Python Software Foundation. (2024). *Documentación de Python 3.13: Tutorial de Python*. https://docs.python.org/es/3.13/tutorial/
- Python Software Foundation. (2024). *Errores y excepciones*. https://docs.python.org/es/3.13/tutorial/errors.html
- Python Software Foundation. (2024). *Módulos y paquetes*. https://docs.python.org/es/3.13/tutorial/modules.html
- Microsoft. (2026). *Python in Visual Studio Code*. https://code.visualstudio.com/docs/languages/python
- UTN – TUPaD. (2026). *Programación 1: enunciado y rúbrica del Trabajo Práctico Integrador*. Material de la cátedra.

7. Conclusiones

> El sistema cumple las siete opciones del menú y resuelve cada situación de error con el mecanismo que pide el enunciado. Lo que más me sirvió fue entender la diferencia entre capturar un error **dentro** de la función para reintentar (opción 1) y dejarlo **salir** hacia main() para cancelar la operación (opciones 3, 5 y 6). También aprendí a evitar colecciones extra: con un contador y con variables que apuntan a elementos del inventario resolví lo mismo sin romper la regla.

Trabajar por partes, probando cada opción antes de seguir, me permitió encontrar los errores enseguida. Por ejemplo, una sangría mal ubicada que funcionaba "por casualidad" y que corregí para que la lógica quede como la diseñé.

Anexo: capturas de las pruebas

```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: abc
Error: invalid literal for int() with base 10: 'abc'

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 9
Error: Opción fuera de rango (1 a 7).

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
```

Menú: abc y 9

```
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 9
Error: Opción fuera de rango (1 a 7).

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 2

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 7
Gracias por usar el sistema. ¡Hasta luego!
PS C:\Users\fanny\Documents\TRABJOS UTN\TP INTEGRADOR FINAL>
```

Menú: 2 y 7

```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 1
¿Cuántas herramientas desea cargar?: abc
Error: invalid literal for int() with base 10: 'abc'
¿Cuántas herramientas desea cargar?: 0
Error: La cantidad debe ser mayor que 0.
¿Cuántas herramientas desea cargar?: 2

Herramienta 1 de 2
Nombre:
Error: El nombre no puede estar vacío.
Nombre: Martillo
Stock inicial: -3
Error: El stock no puede ser negativo.
Stock inicial: x
Error: invalid literal for int() with base 10: 'x'
Stock inicial: 10
"Martillo" cargada con 10 unidades.

Herramienta 2 de 2
Nombre: martillo
Error: Esa herramienta ya existe en el inventario.
Nombre: pinza
Stock inicial: 2
"pinza" cargada con 2 unidades.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
```

Opción 1: reintentos y duplicado

```
RABJOS UTN/TP INTEGRADOR FINAL/main.py"'

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 1
¿Cuántas herramientas desea cargar?: 2

Herramienta 1 de 2
Nombre: pala
Stock inicial: martillo
Error: invalid literal for int() with base 10: 'martillo'
Stock inicial: 2
"pala" cargada con 2 unidades.

Herramienta 2 de 2
Nombre: martillo
Stock inicial: 4
"martillo" cargada con 4 unidades.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 2

--- INVENTARIO ---
pala: 2 unidades
martillo: 4 unidades
```

Opción 1 (stock inválido) y opción 2

```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 3
Todavía no hay herramientas cargadas.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 1
¿Cuántas herramientas desea cargar?: 1

Herramienta 1 de 1
Nombre: Martillo
Stock inicial: 5
"Martillo" cargada con 5 unidades.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 3
```

Opción 3: inventario vacío

```
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 3
Nombre de la herramienta a consultar:
Error: El nombre no puede estar vacío.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 3
Nombre de la herramienta a consultar: MARTILLO
Stock de Martillo: 5 unidades

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 3
Nombre de la herramienta a consultar: Serrucho
"Serrucho" no está en el inventario.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción:
```

Opción 3: vacío, MARTILLO, Serrucho

```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 4
Todavía no hay herramientas cargadas.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 1
¿Cuántas herramientas desea cargar?: 1

Herramienta 1 de 1
Nombre: pala
Stock inicial: 3
"pala" cargada con 3 unidades.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 4

--- HERRAMIENTAS AGOTADAS ---
No hay herramientas agotadas.
```

Opción 4: vacío y sin agotados

```
--- HERRAMIENTAS AGOTADAS ---
No hay herramientas agotadas.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 1
¿Cuántas herramientas desea cargar?: 2

Herramienta 1 de 2
Nombre: pinza
Stock inicial: 0
"pinza" cargada con 0 unidades.

Herramienta 2 de 2
Nombre: lija
Stock inicial: 0
"lija" cargada con 0 unidades.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 4

--- HERRAMIENTAS AGOTADAS ---
- pinza
- lija

===== CONTROL DE INVENTARIO - FERRETERÍA =====
```

Opción 4: con agotados

```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 5
Nombre del nuevo producto:
Error: El nombre no puede estar vacío.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 5
Nombre del nuevo producto: taladro
Stock inicial: abc
Error: invalid literal for int() with base 10: 'abc'

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 5
Nombre del nuevo producto: taladro
Stock inicial: -2
Error: El stock no puede ser negativo.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
```

Opción 5: errores

```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 5
Nombre del nuevo producto: taladro
Stock inicial: 5
"taladro" agregada con 5 unidades.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 5
Nombre del nuevo producto: TALADRO
Error: Esa herramienta ya existe en el inventario.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 2

--- INVENTARIO ---
taladro: 5 unidades
```

Opción 5: alta y duplicado

```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 6
Nombre de la herramienta: Serrucho
"Serrucho" no está en el inventario.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 6
Nombre de la herramienta: martillo
Tipo de movimiento (1 = venta, 2 = ingreso): abc
Tipo de movimiento inválido. Debe ser 1 o 2.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 6
Nombre de la herramienta: martillo
Tipo de movimiento (1 = venta, 2 = ingreso): 1
Unidades: 0
Error: Las unidades deben ser mayores que 0.
```

Opción 6: inexistente, tipo y 0

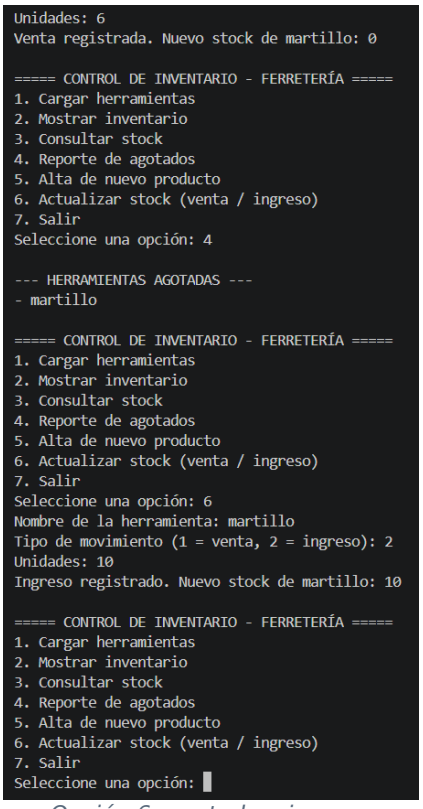
```
===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 6
Nombre de la herramienta: martillo
Tipo de movimiento (1 = venta, 2 = ingreso): 1
Unidades: 8
Error: Stock insuficiente: hay 6 unidades.

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 6
Nombre de la herramienta: martillo
Tipo de movimiento (1 = venta, 2 = ingreso): 1
Unidades: 6
Venta registrada. Nuevo stock de martillo: 0

===== CONTROL DE INVENTARIO - FERRETERÍA =====
1. Cargar herramientas
2. Mostrar inventario
3. Consultar stock
4. Reporte de agotados
5. Alta de nuevo producto
6. Actualizar stock (venta / ingreso)
7. Salir
Seleccione una opción: 4

--- HERRAMIENTAS AGOTADAS ---
```

Opción 6: insuficiente y venta



Opción 6: agotado e ingreso